High-Level Design (HLD) — ASAN VMS

Layihə: ASAN Vətəndaş Müraciət Sistemi (Citizen Appeal Management System) **Versiya:** 1.0.0 **Tarix:** 21 Mart 2026

---

1. Məqsəd və Əhatə Dairəsi

ASAN VMS — vətəndaşların şəhər infrastruktur problemlərini (yollar, kommunal xidmətlər, ictimai təhlükəsizlik və s.) şəkil/video vasitəsilə bildirməsi, süni intellekt ilə avtomatik təhlil edilməsi və admin tərəfindən həllinin yoxlanması üçün veb platformadır.

Əsas funksiyalar:

- Vətəndaş qeydiyyatı (FIN əsaslı) və autentifikasiya
- Şəkil/video yükləmə ilə müraciət göndərmə
- Gemini AI ilə avtomatik kateqorizasiya, prioritetləşdirmə və təsvir yaratma
- EXIF GPS koordinat çıxarılması + HTML5 Geolocation
- Admin paneli ilə müraciət idarəetməsi
- AI əsaslı həll yoxlaması (before/after müqayisə)
- AI-generated şəkil aşkarlanması (EXIF + Gemini)

---

2. Sistemin Ümumi Arxitekturası

```
graph TB
    subgraph "Client Layer"
        CP["👷 Vətəndaş Paneli<br/>React 19 + Vite 7<br/>Port 5173"]
        AP["👮 Admin Paneli<br/>React 19 + Vite 7<br/>Port 5174"]
    end

    subgraph "API Gateway Layer"
        RL["Rate Limiter<br/>Global: 300/15dəq<br/>Auth: 20/15dəq"]
        CORS["CORS Whitelist"]
        HLM["Helmet Security"]
        SAN["NoSQL Sanitizer"]
    end

    subgraph "Application Layer"
        AUTH["Auth Module<br/>JWT + bcrypt"]
        APPEAL["Appeal Module<br/>CRUD + Pagination"]
        ADMIN["Admin Module<br/>Verify + Resolve"]
        UPLOAD["Upload Module<br/>Multer (10MB)"]
    end

    subgraph "AI Service Layer"
        GA["Gemini Analyze<br/>Şəkil → Təhlil"]
        GV["Gemini Verify<br/>Before/After Müqayisə"]
    end

    subgraph "Data Layer"
        DB[(MongoDB<br/>3 Collection)]
        FS["📁 File Storage<br/>uploads/"]
```

```
    end

    subgraph "External Services"
        GEMINI["Google Gemini 2.5 Flash"]
        RLW["Railway Platform"]
    end

    CP -->|HTTP/REST| RL
    AP -->|HTTP/REST| RL
    RL --> CORS --> HLM --> SAN
    SAN --> AUTH
    SAN --> APPEAL
    SAN --> ADMIN
    APPEAL --> UPLOAD
    ADMIN --> UPLOAD
    APPEAL --> GA
    ADMIN --> GV
    GA --> GEMINI
    GV --> GEMINI
    AUTH --> DB
    APPEAL --> DB
    ADMIN --> DB
    UPLOAD --> FS
```

---

3. Komponent Diaqramı

```
graph LR
    subgraph "Backend Components"
        direction TB
        A1["app.js<br/>Entry Point"]
        A2["routes/<br/>authRoutes · appealRoutes"]
        A3["controllers/<br/>appealController · adminController"]
        A4["middleware/<br/>auth · upload"]
        A5["models/<br/>User · Appeal · Media"]
        A6["services/<br/>geminiService"]

        A1 --> A2 --> A3
        A2 --> A4
        A3 --> A5
        A3 --> A6
    end

    subgraph "Admin Panel Pages"
        B1["Login"]
        B2["Dashboard"]
        B3["Users"]
        B4["VerifyAppeal"]
    end

    subgraph "User Panel Pages"
```

```
        C1["Home"]
        C2["Login / Register"]
        C3["SubmitAppeal"]
        C4["MyAppeals"]
        C5["AppealDetail"]
    end
```

---

4. Verilənlər Bazası Dizaynı

```
erDiagram
    USER {
        ObjectId _id PK
        String email UK
        String fin UK
        String password
        String role "citizen | admin"
        String firstName
        String lastName
        Date createdAt
        Date updatedAt
    }

    APPEAL {
        ObjectId _id PK
        ObjectId citizenId FK
        ObjectId initialMediaId FK
        String status "Pending Review | Resolved | Mismatch | Rejected"
        String title
        String description
        String category
        String priority "Low | Medium | High | Critical"
        Object location
        Object confidence_scores
        Object verification
        String rejectionReason
        Date createdAt
        Date updatedAt
    }

    MEDIA {
        ObjectId _id PK
        String originalName
        String filename
        String url
        String mimeType
        Number size
        ObjectId uploadedBy FK
        ObjectId appealId FK
        String type "before | after | other"
        Date createdAt
```

```
    }

    USER ||--o{ APPEAL : "citizenId"
    USER ||--o{ MEDIA : "uploadedBy"
    APPEAL ||--|| MEDIA : "initialMediaId"
    APPEAL ||--o| MEDIA : "verification.resolutionMediaId"
    MEDIA }o--|| APPEAL : "appealId"
```

---

5. API Endpoint Xəritəsi

Autentifikasiya

Method	Endpoint	Giriş	Təsvir
POST	`/api/auth/register`	Açıq	Yeni istifadəçi qeydiyyatı
POST	`/api/auth/login`	Açıq	FIN və ya email ilə giriş

İstifadəçi İdarəetməsi (Admin)

Method	Endpoint	Giriş	Təsvir
GET	`/api/auth/users`	Admin	Bütün admin istifadəçilərinin siyahısı
POST	`/api/auth/users`	Admin	Yeni admin yaratma
PUT	`/api/auth/users/:id`	Admin	İstifadəçi redaktəsi
DELETE	`/api/auth/users/:id`	Admin	İstifadəçi silmə

Müraciətlər

Method	Endpoint	Giriş	Təsvir
POST	`/api/appeals/analyze`	Vətəndaş	Media yükləmə → AI təhlil
POST	`/api/appeals`	Vətəndaş	Müraciətin təsdiqi
GET	`/api/appeals`	Auth'd	Müraciətlər siyahısı (paginasiya ilə)
GET	`/api/appeals/:id`	Auth'd	Tək müraciət detalları
PUT	`/api/appeals/:id`	Admin	Müraciət redaktəsi
POST	`/api/appeals/:id/verify`	Admin	Həll yoxlaması (AI müqayisə)
POST	`/api/appeals/:id/resolve`	Admin	Müraciətin bağlanması

---

6. Əsas İş Axınları

6.1 Müraciət Göndərmə Axını

```
sequenceDiagram
    actor V as Vətəndaş
    participant FE as User Panel
    participant API as Backend API
    participant AI as Gemini AI
    participant DB as MongoDB
    participant FS as File Storage

    V->>FE: Şəkil çəkir / yükləyir
    FE->>API: POST /api/appeals/analyze (media)
    API->>FS: Faylı saxlayır
    API->>API: EXIF GPS çıxarır
    API->>AI: Şəkili göndərir
    AI-->>API: JSON (title, category, priority, description)
    API->>DB: Media record yaradır
    API-->>FE: AI təhlil nəticəsi + mediaId
    FE-->>V: Önizləmə göstərir
    V->>FE: Təsdiq edir
    FE->>API: POST /api/appeals (mediaId + aiAnalysis)
    API->>DB: Appeal yaradır
    API-->>FE: Uğurlu cavab
```

6.2 Həll Yoxlama Axını

```
sequenceDiagram
    actor A as Admin
    participant FE as Admin Panel
    participant API as Backend API
    participant AI as Gemini AI
    participant DB as MongoDB

    A->>FE: Həll şəklini yükləyir
    FE->>API: POST /api/appeals/:id/verify (resolutionMedia)
    API->>API: EXIF AI-generated yoxlama
    API->>AI: Before + After şəkilləri göndərir
    AI-->>API: JSON (same_location, issue_resolved, is_ai_generated)
    API->>DB: Verification nəticəsini saxlayır
    API-->>FE: Yoxlama nəticəsi

    alt Uğurlu yoxlama (mismatch_warning = false)
        A->>FE: "Həll et" düyməsinə basır
        FE->>API: POST /api/appeals/:id/resolve
        API->>DB: Status → "Resolved"
    else Uyğunsuzluq (mismatch_warning = true)
        A->>FE: Yenidən cəhd və ya rədd edir
    end
```

6.3 Autentifikasiya Axını

```
sequenceDiagram
    actor U as İstifadəçi
    participant FE as Frontend
    participant API as Backend
    participant DB as MongoDB

    U->>FE: FIN/email + parol daxil edir
    FE->>API: POST /api/auth/login
    API->>DB: İstifadəçini tapır
    API->>API: bcrypt.compare(password)
    alt Uğurlu
        API-->>FE: JWT Token + user info
        FE->>FE: localStorage-ə saxlayır
        FE-->>U: Dashboard-a yönləndirir
    else Uğursuz
        API-->>FE: 401 ERR_INVALID_CREDS
        FE-->>U: Xəta mesajı
    end
```

---

7. Təhlükəsizlik Arxitekturası

```
graph TD
    REQ["Gələn Sorğu"] --> RL["Rate Limiter<br/>Global: 300/15dəq<br/>Auth: 20/15dəq"]
    RL --> CORS["CORS<br/>Whitelist doğrulama"]
    CORS --> HLM["Helmet<br/>HTTP başlıq təhlükəsizliyi"]
    HLM --> SAN["NoSQL Sanitizer<br/>$ və . simvollarının filtrasiyası"]
    SAN --> JWT["JWT Doğrulama<br/>Bearer Token"]
    JWT --> ROLE["Rol Yoxlaması<br/>citizen | admin"]
    ROLE --> CTRL["Controller"]

    style RL fill:#ff6b6b,color:#fff
    style CORS fill:#ffa94d,color:#fff
    style HLM fill:#ffd43b,color:#000
    style SAN fill:#69db7c,color:#000
    style JWT fill:#4dabf7,color:#fff
    style ROLE fill:#845ef7,color:#fff
```

Qat	Mexanizm	Detallar
Şəbəkə	Rate Limiting	express-rate-limit, IP əsaslı
Transport	CORS	Origin whitelist, credentials dəstəyi
HTTP	Helmet	XSS, clickjacking, MIME sniffing qoruması
Input	Sanitization	NoSQL injection qarşısının alınması
Auth	JWT + bcrypt	30 günlük token, 10 round salt
AI Fraud	EXIF + Gemini	AI-generated şəkil aşkarlanması

8. Deployment Arxitekturası

```
graph TB
    subgraph "Railway Platform"
        subgraph "Backend Service"
            BE["Docker Container<br/>node:20-alpine<br/>Port: 5000"]
            HC["/health endpoint"]
        end
        subgraph "Admin Service"
            ADMIN["Docker Container<br/>serve -s dist<br/>Port: 3000"]
        end
        subgraph "User Service"
            USER["Docker Container<br/>serve -s dist<br/>Port: 3000"]
        end
        subgraph "Database"
            MONGO["MongoDB<br/>(Railway Plugin)"]
        end
    end

    ADMIN -->|VITE_API_URL| BE
    USER -->|VITE_API_URL| BE
    BE --> MONGO
    HC -.->|healthcheck| BE

    style BE fill:#339af0,color:#fff
    style ADMIN fill:#20c997,color:#fff
    style USER fill:#20c997,color:#fff
    style MONGO fill:#51cf66,color:#fff
```

Xidmət	Baza Image	Build Strategiyası	Port
Backend	node:20-alpine	Single-stage, `npm install --omit=dev`	5000
Admin	node:20-alpine	Multi-stage (build → serve)	3000
User	node:20-alpine	Multi-stage (build → serve)	3000

---

9. Texnologiya Stack Xülasəsi

Qat	Texnologiya	Versiya
Runtime	Node.js	20
Backend	Express	5.2
Database	MongoDB + Mongoose	9.2
AI	Google Gemini (@google/genai)	2.5 Flash
Auth	JWT + bcryptjs	9.0 / 3.0

Frontend	React + React Router	19.2 / 7.13
Build	Vite	7.3
CSS	Tailwind CSS	4.2
Upload	Multer	2.0
Security	Helmet + express-rate-limit	8.1 / 8.3
Deployment	Railway + Docker	—

---

10. Qeyri-funksional Tələblər

Tələb	Cari Həyata Keçirilmə
Performans	Pagination (max 100/sorğu), rate limiting
Təhlükəsizlik	6-qat middleware zənciri, bcrypt hashing
Miqyaslanma	Docker konteynerləri, stateless JWT
Mövcudluq	Railway health check, `on_failure` restart
Dil dəstəyi	Tam Azərbaycan dili (AI cavabları daxil)
Fayl limiti	10 MB, JPEG/PNG/WebP/MP4

---

11. Məhdudiyyətlər və Gələcək Təkmilləşdirmələr

#	Məhdudiyyət	Təklif olunan Həll
1	Fayl saxlama ephemeral (Railway deploy-da silinir)	AWS S3 / Cloudinary inteqrasiyası
2	Bildiriş sistemi yoxdur	Email/SMS notification əlavə etmə
3	Avtomatik testlər yoxdur	Jest + Supertest (backend), Vitest (frontend)
4	Validasiya manual	Joi/Zod validasiya kitabxanası
5	Böyük komponentlər (VerifyAppeal: 35KB)	Komponent dekompozisiyası
6	`.env` faylında açıq sirrlər	Environment variable rotation, `.gitignore`